UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INFORMATICA Y SISTEMAS

PROYECTO FINAL

MANUAL TECNICO

INTEGRANTES:	**Carne**
CRISTHIAN ESTUARDO DE LEON PEREZ	1540225
MAYNOR EDUARDO MORALES CHANG	1553025
SANTOS JUAN DIEGO CHUC GUTIÉRREZ	1532825

QUETZALTENANGO, NOVIEMBRE 2026

CAMPUS SAN ALBERTO HURTADO, S.J

1. IMPLEMENTACION Y COMPLEJIDAD

- **Grafo:**
  Implementación: la estructura de nuestro grafo se encuentra internamente haciendo uso de listas de adyacencia para optimizar la iteración sobre las conexiones directas de cada nodo, lo que hace fácil una consulta entre vecinos (vértices) de manera eficiente y evitando el alto consumo de la memoria que necesitaría si fuera una matriz de adyacencia en redes de alta dispersión.

  Justificación: De lo que se encarga el grafo en nuestro sistema es de modelar la red geográfica de la distribución de donde las ciudades, sucursales o centros de entrega actúan como vértices y las rutas que toman como las aristas que estas mismas tienen un costo.

  Complejidad Big-O: Depende de los Vértices y las Aristas O(V + A), y la inserción de vértices y aristas es de O(1).

- **Dijkstra:**
  Implementación: Esta estructura en nuestro sistema se apoya de una cola de prioridad para que así se pueda extraer el nodo no visitado con la distancia acumulada menor, evitando revisiones de líneas que no son necesarias y asi poder optimizar la coincidencia del algoritmo.

  Justificación: Lo que realiza esta estructura básicamente es resolver el problema de encontrar la ruta mas corta desde el punto de origen hasta el destinatario del paquete, esto nos permite minimizar los costos de traslado.

  Complejidad Big-O (): Gracias al uso de la cola de prioridad el espacio se vuelve de O(V) ósea que depende del Vértice.

- **Recorridos DFS y BFS:**
  Implementación: DFS es una estructura que se aplicó con recursividad para explorar a profundidad cada uno de los caminos encontrados hasta que se agote su rama antes de que este regrese, Y el BFS se implementa una estructura de tipo cola usando Queue<T> que es una librería que nos brinda C#, esto para poder recorrer los niveles por anchura.

  Justificación: El recorrido BFS nos permite encontrar rutas con el menor número de saltos posibles o también que explore la red por capas de proximidad, mientras que el DFS se utiliza para verificar la conectividad total y analizar cada componente conexo en el sistema vial.

Complejidad Big-O (): Tiempo O(V + A) al procesar cada Vértice y Arista a lo mucho una sola vez; Espacio O(V) por el consumo de memoria de la cola o de llamadas recursivas.

- **Trie:**
  Implementación: Esta estructura organiza cada carácter de cada destino u origen de una forma jerárquica en nodos hijos para así permitir filtrar y localizar subconjuntos masivos prefijadas carácter por carácter sin necesidad de recorrer de linealmente toda la base de datos de palabras.

  Justificación: Esta estructura fue diseñado para optimizar las búsquedas de texto o un autocompletado rápido a los nombres de cada nombre de departamento o municipio y así también de códigos de seguimiento.

  Complejidad Big-O: Inserción y Búsqueda en O(L), donde L significa la longitud de la cadena evaluada.

- **Tabla Hash:**
  Implementación: La estructura esta aplicada en una función Hash donde esta se encarga de transformar la clave de búsqueda en un índice para ser almacenado directamente dentro del almacenamiento de un arreglo interno, donde este mismo maneja las posibles colisiones que pueden llegar a existir.

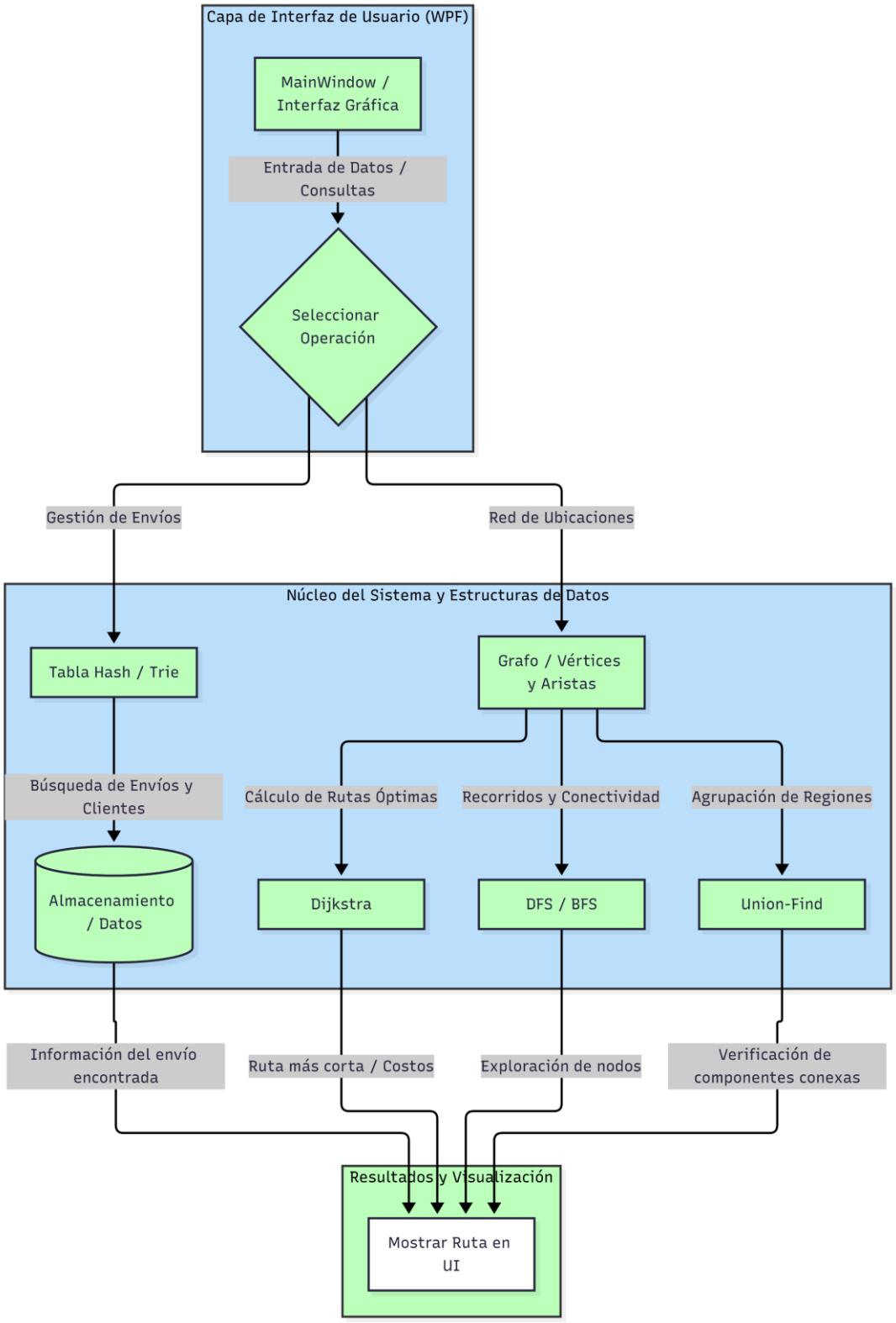
  Justificación: Esta estructura es fundamental para gestionar los registros de envíos donde se requiere buscar, insertar y actualizar la información de forma rápida e inmediata utilizando código único para cada registro.

  Complejidad Big-O (): En la búsqueda, inserción y eliminación cuentan con O(1) y en caso de colisiones seria O(N); Espacio O(N) según la cantidad de elementos que tengamos almacenados.

- **Unión-Find:**
  Implementación: Esta estructura incorpora optimizaciones clave como compresión de caminos (path compression) y unión por rango (union by rank), aplanando la jerarquía de los árboles en cada consulta para mantener tiempos de respuesta casi constantes.

  Justificación: Administra particiones de conjuntos disjuntos para comprobar rápidamente si dos nodos ya se encuentran conectados dentro de la misma red logística o componente conexa.

2. DIAGRAMA DE FLUJO



3. DIAGRAMA DE CLASES

